

The Engine and Stack Guide

Everything you need to know to write, run, test and ship a game on this engine yourself. This file lives on `main` and is part of every game branch (you get it by merging `main`). Each game also has its own guide: `docs/ROCKFALL.md`, `docs/FRUITSLICE.md`.

Contents: [1 The big picture](#) · [2 The stack](#) · [3 Folder tour](#) · [4 How a frame works](#) · [5 Writing a game](#) · [6 Engine services](#) · [7 Day-to-day workflow](#) · [8 Tests](#) · [9 Git: branches and worktrees](#) · [10 Share a playable link](#) · [11 Phones: Android and iOS](#) · [12 Ads \(AdMob\)](#) · [13 Pitfalls](#) · [14 Glossary](#)

1. The big picture

A game made here has three layers. Keep them apart and everything stays easy to test and change.

<code>src/main.ts</code> <code>boot(createGame)</code> - joins the two sides	
<code>src/engine/</code> (same in every	<code>src/game/</code> (different per
game, lives on `main`)	game, lives on its branch)
- game loop	- <code>sim/</code> the rules (pure)
- pointer / touch input	- <code>view/</code> what is drawn
- <code>PixiJS</code> canvas, letterbox	- <code>ui/</code> menus, HUD (DOM)
- ads, haptics, audio,	- <code>index</code> the flow: which
analytics, storage	screen, when
- draws NOTHING itself	

- **Engine** = tools. It loads libraries, runs the clock, gives you a canvas and services. Run on its own (`main`) it shows a blank black screen. That is correct.
- **Game** = everything the player sees and does: menus, score, rules, art, sounds' meaning.
- **Sim** (`src/game/sim`) = the rules written as plain functions on plain data. No drawing, no browser, no clock, no `Math.random`. Because it is pure it can be unit-tested in milliseconds and replayed exactly from a seed.

The rule that holds it together: **dependencies only point one way**: `main.ts` → `game` → `engine`, and inside the game `index` (flow) → `ui / view` → `sim`. `tests/architecture.test.ts` fails if you break it.

2. The stack: what each technology is for

Tool	What it is	Why it is here / what you do with it
TypeScript 7 (strict)	JavaScript with types	Catches mistakes before running. <code>npm run typecheck</code> . Strict settings: no implicit <code>any</code> , array reads may be <code>undefined</code> (<code>noUncheckedIndexedAccess</code> , so write <code>arr[i]!</code> only when sure), <code>verbatimModuleSyntax</code> (write <code>import type { X }</code> for types).
Vite 8	Dev server + bundler	<code>npm run dev</code> serves the game with instant reload; <code>npm run build</code> makes the <code>dist/</code> folder you can upload anywhere. Config: <code>vite.config.ts</code> (<code>base: './'</code> so the build works from any sub-folder).
PixiJS v8	2D renderer (WebGPU, falls back to WebGL)	Draws sprites and vector shapes (<code>Graphics</code>) on a <code><canvas></code> . Only files in <code>view/</code> import it. You build a tree of <code>Container</code> s under <code>view.field</code> .
DOM + CSS	Normal web page elements	Menus, buttons, HUD text are plain HTML in an overlay (<code>#ui</code>) above the canvas. Much easier than drawing text and buttons in Pixi, and accessible.
Capacitor 8	Wraps the web build in a native app	Turns <code>dist/</code> into an Android Studio / Xcode project (<code>android/</code> , <code>ios/</code>). Config: <code>capacitor.config.json</code> (app name + app ID).
@capacitor-community/admob	AdMob plugin	Rewarded + interstitial ads on devices. In a browser a fake ad service stands in.
@capacitor/haptics	Vibration	<code>haptics.impact('light')</code> etc. No-op where unsupported.
Web Audio	Browser sound API	The engine <i>synthesises</i> sound effects (no audio files).
Vitest 5	Unit-test runner	<code>npm test</code> . Tests live in <code>tests/</code> .
Playwright	Drives a real browser	<code>npm run e2e</code> plays the game like a user and saves screenshots to <code>e2e/screenshots/</code> . Emulates a Pixel 7 phone.
Node ≥ 22.12	Runs all the tooling	Older Node will not run Vite 8 / Vitest 5 / the Capacitor CLI.
Git + worktrees	Version control	<code>main</code> = engine, one branch per game, each checked out in its own folder.

You do **not** need a framework (React, etc.). The game is a plain TypeScript program.

3. Folder tour

index.html	the page: #game (canvas goes here) and #ui (menus go here)
capacitor.config.json	app ID + name for the native apps (CHANGE per game)
vite.config.ts	build settings
package.json	commands (scripts) and dependencies
.env.production.example	copy to .env.production for real AdMob IDs
scripts/setup-native.mjs	patches the generated android/ios projects (ad app IDs, portrait lock)
src/main.ts	boot(createGame)
src/engine/	
boot.ts	starts everything, builds the EngineContext, runs the loop
context.ts	THE CONTRACT: EngineContext (what you get) and Game (what you return)
core/	
loop.ts	fixed-timestep loop
input.ts	pointer position + drag gestures
layout.ts	fits the 360x640 world into any window (letterbox)
rng.ts	seeded random numbers (mulberry32)
storage.ts	safe localStorage helpers
config.ts	WORLD = 360 x 640
services/	ads.ts, fakeAds.ts, admobAds.ts, analytics.ts, haptics.ts, audio.ts
view/	
view.ts	the Pixi canvas, `field` layer, clientToWorld
ui/	
stage.ts, base.css	sizes a DOM stage exactly over the playfield; safe-area insets
src/game/	YOUR GAME (empty on main)
tests/	Vitest unit tests (+ architecture rules)
e2e/	Playwright tests (+ screenshots/)
android/ ios/	generated native projects - exist only on game branches
.vscode/	tasks + launch configs (Run, Build, Test, Debug in Chrome)
.github/workflows/	pages.yml - publishes playable links
docs/	these guides

4. How a frame works

Fixed timestep. The simulation advances in equal steps (usually 1/60 s) no matter how fast the screen refreshes, so the game plays identically on a 60 Hz and a 144 Hz phone, and runs can be replayed from a seed. `FixedLoop` collects real elapsed time, runs `game.update(step)` as many times as needed, then calls `game.render(alpha)` once, then the engine draws the frame.

```
each animation frame:
  accumulate elapsed time (capped at 0.25 s so a pause cannot cause a burst)
  while accumulator >= step: game.update(step)      <- the simulation, deterministic
  game.render(alpha)      alpha = leftover / step <- blend previous and current positions
  view.draw()             <- Pixi paints
```

Interpolation. Because `update` runs at a fixed rate, objects store `prevX/prevY` (position at the previous step) and `render(alpha)` draws `lerp(prev, current, alpha)`. That is what makes motion smooth at any refresh rate.

The world. All game coordinates are in a fixed **360 × 640** world (portrait 9:16). The engine scales it to fit the window and draws black bars if the window is a different shape (letterbox). `view.clientToWorld(x, y)` converts a screen/pointer position to world units. DOM UI sized with the `--u` CSS variable (CSS pixels per world unit) lines up with the canvas.

Two layers on screen. Canvas (Pixi) for the playfield, DOM overlay (`#ui`) for menus and HUD. The HUD must have `pointer-events: none` so it never steals a drag; buttons get pointer events. Drag gestures from `input.onGesture` only start on the canvas, never on a button.

5. Writing a game, step by step

A game is one function:

```
// src/game/index.ts
import type { GameFactory } from '../engine';

export const createGame: GameFactory = (ctx) => {
  // ctx: world, input, view, ads, analytics, haptics, audio, params, uiRoot, resetClock
  return {
    step: 1 / 60,
    update(dt) { /* advance the simulation by one fixed step */ },
    render(alpha) { /* move display objects to match the state */ },
  };
};
```

The recommended shape (both RockFall and Fruit Slice follow it):

1. `sim/config.ts` - every tuning number in one object (`CONFIG`). Tweak the feel here only.
2. `sim/*.ts` - state as a plain object, and functions `createState(seed)`, `step(state, input)`. Randomness only from `state.rng` (seeded). Things that happened are pushed to `state.events` (`{type: 'died'}`, `{type: 'cut', ...}`) and the flow drains them: the sim never calls the UI.
3. `view/scene.ts` - `createScene(view.field)` returns `update(frame)`. It reads the state and moves Pixi objects; it never changes the state. Create display objects when something first appears (keyed by id in a `Map`), update them every frame, destroy them when they disappear.
4. `ui/ui.ts` - builds the DOM screens into `ctx.uiRoot`. It knows nothing about game state: it gets plain values in (`setScore(12)`) and calls callbacks out (`onPlay`).
5. `index.ts` (the *flow*) - the phase machine (`menu` → `playing` → `paused` → `over`), connects the pieces: feeds input to the sim, drains events into the scene/UI/sound/ads, saves best scores.
6. **Tests** - a unit test for every rule in `tests/`, and an e2e spec that plays it.

Identity of a new game: change `capacitor.config.json` (`appId`, `appName`), `package.json` name, the `<title>` in `index.html`, `public/favicon.svg`, and the `GAME_ID` you use to prefix saved data.

Pointer input

```
ctx.input.onGesture((g) => { // g.phase: 'start' | 'move' | 'end' | 'cancel'
  const p = ctx.view.clientToWorld(g.clientX, g.clientY); // -> {x, y} in world units
});
ctx.input.clientX // polled position of the pointer (may be null before the first event)
```

Seeded randomness

```
import { createRng, nextFloat, nextRange } from '../engine/core/rng';
const rng = createRng(seed); // plain object, can live in your state
nextFloat(rng); // 0..1
nextRange(rng, 10, 20); // 10..20
```

Same seed = same sequence = reproducible bugs. In the browser add `?seed=42` (both games read it).

6. The services you can use

All services are safe: they never throw into your code.

Service	Use
<code>ads.isRewardedReady()</code> , <code>await ads.showRewarded()</code> → <code>true</code> if the player earned the reward; <code>ads.preloadRewarded()</code>	"Watch an ad to continue / double coins". Check <code>isRewardedReady()</code> to decide whether to show the button.
<code>ads.isInterstitialReady()</code> , <code>await ads.showInterstitial()</code> , <code>ads.preloadInterstitial()</code>	Short ad between screens. You decide when (never right after a rewarded ad; rate-limit it).
<code>haptics.impact('light' 'medium' 'heavy')</code> , <code>haptics.notify('success' 'warning' 'error')</code> , <code>haptics.setEnabled(bool)</code>	Vibration on hits/cuts and results.
<code>audio.tone({...})</code> , <code>audio.noise({...})</code> , <code>audio.setEnabled(bool)</code> (<code>unlock()</code> is called by the engine on the first touch)	Synthesised sound: tones (beeps, sweeps) and noise (swishes, bursts). A game builds its named effects on top (Fruit Slice: <code>src/game/sfx.ts</code> , read it for examples).
<code>analytics.track('event', {...})</code>	Logs events (console in dev). Swap the implementation in <code>services/analytics.ts</code> for a real backend later.
<code>safeGetItem/safeSetItem/safeGetNumber/safeSetNumber</code> (<code>engine/core/storage</code>)	<code>localStorage</code> that never throws (private mode, quota). Prefix keys with your game id.

Service	Use
<code>params.get('seed')</code>	URL query flags. Engine itself reads <code>?ads=no-fill</code> / <code>?ads=skip</code> (fake ad failures).
<code>resetClock()</code>	Call after un-pausing so the loop does not see a big time gap.

In a **browser** ads are a fake service (a "Test ad" overlay). On a **device** the same calls go to AdMob. Your game code is identical.

7. Day-to-day workflow

Install Node 22+, then in the game's folder:

```
npm install          # once
npm run dev          # play at http://localhost:5173 (also reachable on your Wi-Fi from a phone)
npm run check        # typecheck + all unit tests. Run after EVERY change.
npm run build        # production build into dist/
npm run e2e          # Playwright plays the game; then OPEN e2e/screenshots/*.png and look
```

- Set `CHROMIUM_PATH` to your Chrome (`C:\Program Files\Google\Chrome\Application\chrome.exe`) if Playwright has no browser of its own.
- **Passing tests do not prove the canvas drew anything.** After any visual change, look at the screenshots.
- Useful dev URL flags: `?seed=N` , `?ads=no-fill` ; each game documents its own (`?time=` , `?gallery` ...).
- In dev builds `window.__engine` and `window.__game` expose read-only snapshots (that is what the e2e tests read). They do not exist in production builds.
- **VS Code:** open the folder, click *Trust*. `Terminal` → `Run Task...` lists "dev: run in browser", build, "check", unit tests, e2e tests and the Android/iOS tasks; `F5` launches the game in Chrome with the debugger attached (breakpoints in `.ts` files work).
- **Windows PowerShell** may block `npm.ps1` ("running scripts is disabled"): run once `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` , Or use `npm.cmd` .

8. Tests

- **Unit (Vitest, `tests/`):** pure logic. The sim is pure, so you can build a state, call `step()` 600 times and assert on the result. Fast (<1 s for hundreds).
- **Architecture (`tests/architecture.test.ts`):** enforces the layering (see CLAUDE.md for the list). If it fails you broke a dependency rule; fix the code, not the test.
- **End to end (Playwright, `e2e/`):** starts a dedicated dev server on its own port (never reuses yours on 5173), opens a Pixel 7 sized page, clicks and drags like a player, saves screenshots.
`e2e/engine.spec.ts` tests the engine itself and is identical in every game.
- Habit: new rule → unit test first/with it; new screen → e2e + look at the screenshot.

9. Git: main is the engine, every game is a branch

main	= the engine (no game).	folder: Engine_Test
rockfall	= a game built on main.	folder: RockFall (git worktree)
fruitslice	= a game built on main.	folder: FruitSlice (git worktree)

A **worktree** is a second folder showing a different branch of the same repository, so you never "switch branches" in one folder - every game is simply open in its own folder (and VS Code window).

Rules we follow:

1. **Engine fixes happen on main first**, proven there (`npm run check` , `npm run e2e`), then merged into each game: `cd ../RockFall && git merge main && npm run check` .
2. **Never merge a game into main** . Never edit `src/engine/` inside a game branch; if a game needs something new from the engine, add a hook on `main` , then merge.
3. Keep engine commits and game commits separate.
4. Never rewrite pushed history; push only when you mean to.

Start a new game

```
cd Engine_Test # the main folder
git worktree add -b mygame ../MyGame main # new branch + new folder
cd ../MyGame
npm install
# edit capacitor.config.json (appId/appName), package.json name, index.html title, then:
npm run dev
```

Then write `src/game/` as in section 5. Commit on the `mygame` branch.

Handy git commands

```
git status # what changed
git add -A && git commit -m "message"
git log --oneline # history
git worktree list # the folders and which branch each shows
git merge main # bring engine updates into the game you are in
git push # publish the current branch (GitHub Desktop does the same)
```

10. Share a playable link (GitHub Pages)

`.github/workflows/pages.yml` (on `main` , merged into the games) builds **every game branch** on GitHub and publishes them as web pages. Ads are fake in a browser and saves are local to each player's own browser, so it is safe to hand around.

One-time setup, on github.com:

1. Push `main` and each game branch (GitHub Desktop → *Push origin*, per branch).
2. Repository → **Settings** → **Pages** → **Build and deployment** → **Source: GitHub Actions**.
3. Repository → **Actions** tab → "Playable builds" → *Run workflow* (or just push again).
4. After ~2 minutes your links are:

```
https://<your-github-name>.github.io/<repo-name>/          list of all games
https://<your-github-name>.github.io/<repo-name>/rockfall/
https://<your-github-name>.github.io/<repo-name>/fruitslice/
```

(For this repo: `https://max-martin-sherie.github.io/Engine_Test/...`) Every later push to `main`, `rockfall` or `fruitslice` republishes everything. A new game branch appears automatically once it is pushed (add its name to `on.push.branches` in the workflow if you want pushes to that branch to trigger a rebuild too). If the repository is private, Pages needs a paid GitHub plan. Friends open the link on their phone; "Add to Home Screen" makes it feel like an app.

11. Phones: Android and iOS

The web build is wrapped by Capacitor. Native projects are **generated** and live only on game branches (never on `main`). Never hand-edit `android/` or `ios/`: change `scripts/setup-native.mjs` (it is idempotent and tested) and re-run it.

Android (Windows/Mac/Linux): install Android Studio (brings the SDK) and **JDK 21**.

```
npm run android:setup    # once: build, `cap add android`, patch manifest (test ad app ID,
                           portrait)
npm run android          # build, sync, run on a connected phone/emulator
```

iOS needs a Mac with Xcode: `npm run ios:setup`, then `npm run ios`.

Every web change needs a rebuild + sync (`npm run android` does both). The app ID in `capacitor.config.json` (e.g. `com.maxmartin.fruitslice`) is permanent once published: it identifies the app in the stores and in AdMob.

Before a store release you also need: app icons + splash screen (Capacitor docs: `@capacitor/assets`), a signed release build (Android Studio *Build* → *Generate Signed Bundle*), a privacy policy URL (ads need one), and the store listing. Test on a real device: touch feel, ads, vibration and sound cannot be verified in a desktop browser.

12. Ads (AdMob)

- **Development = Google's test ads**, always. `VITE_ADS_TESTING` defaults to testing.
- For a release: AdMob console → add the app → create a **Rewarded** unit (and **Interstitial** if the game uses one) per platform. Copy `.env.production.example` to `.env.production`, set `VITE_ADS_TESTING=false` and the unit IDs. A missing real ID disables that ad kind (it never falls back to a test ID). Put the AdMob **app ID** into the native project via `scripts/setup-native.mjs`.

- Consent (EU) and iOS App Tracking Transparency prompts are handled by the engine's AdMob service.
- Rules that make ads behave (details in `CLAUDE.md`): ask for consent after SDK init; trust ad *events*, not promises, for rewarded outcomes; only one full-screen ad at a time; don't show an interstitial right after a rewarded ad.

13. Pitfalls we already hit

- **Old Node:** Vite 8 / Vitest 5 / Capacitor 8 need Node ≥ 22 . Check `node -v`.
- `capacitor.config.ts` **does not work with TypeScript 7** - it is `capacitor.config.json` on purpose.
- **Writing files from a shell:** backticks inside double quotes run as commands; use the editor.
- **CRLF line endings on Windows** broke a fixture test; `.gitattributes` forces LF.
- `.focus()` **on buttons** shows a focus ring on touch - never call it.
- **A HUD without** `pointer-events: none` swallows drags.
- **Pixi's accessibility button** appears on phones; the engine removes it after init.
- **Vite dev server treats URLs ending in** `.json` **as files** - put query parameters first.
- **E2E reusing your dev server** tested the wrong folder once; e2e always uses its own port (5199).
- `Math.random` **in the sim** breaks replays and tests; use `state.rng`.
- **Rewarded ads:** the promise from the plugin can hang on Android; only the events are reliable.

14. Glossary

- **World units:** the game's own coordinate system (360×640). Not pixels.
- **Tick / step:** one fixed simulation update (1/60 s).
- **Alpha:** how far we are between two steps, used to smooth rendering.
- **Sim:** the pure rules code. **View:** Pixi drawing. **UI:** DOM menus. **Flow:** the glue (`index.ts`).
- **Seed:** a number that fixes the random sequence of a run.
- **Letterbox:** black bars when the window shape differs from 9:16.
- **Worktree:** an extra folder for another branch of the same repo.
- **Capacitor:** the tool that wraps a web app into a native Android/iOS app.
- **Rewarded ad:** an ad the player chooses to watch for a reward. **Interstitial:** a short full-screen ad between screens.
- **e2e:** end-to-end test, a robot plays the real game in a real browser.